

Judgment Fiber Bundles: Trust as a Geometric Connection in Multi-Agent AI Verification

JuGeo Research Group

March 24, 2026

Abstract

Multi-agent AI systems now routinely decompose tasks across several tool-using language models, but they still lack formal foundations for deciding when a collection of agent outputs should be trusted as a whole. Existing techniques such as majority vote, debate, self-reflection, and pairwise adjudication are fundamentally local: they compare outputs one edge at a time and therefore miss structural team pathologies. We introduce the *judgment fiber bundle*, a construction at the intersection of differential geometry, sheaf theory, and formal verification. A judgment (σ, e, τ, χ) is a claim together with its evidence, trust level, and evidence channel. These judgments form the fibers of a bundle $\pi : \mathcal{E} \rightarrow \mathcal{B}$ over an agent-task site. Trust is not treated as a flat label attached to an output; instead, trust transport between agents defines a connection ∇ on the bundle. Its curvature detects structural team unreliability invisible to pairwise checks, while its holonomy detects loop-level defects such as trust monopolies, echo chambers, and cascades. The first Chern class c_1 provides a computable global invariant: $c_1 = 0$ if and only if a globally consistent trust assignment exists. We show that the judgment bundle strictly generalizes Čech cohomological consistency checking: H^1 contradictions embed as localized curvature, H^2 cascades as holonomy, and phantom consensus as flat-but-evidentially-empty regions. We describe an implementation in **jugeo-agents**, including adapters for CLAUDE CODE, COPILOT CLI, and CODEX, and present coding and web-stack examples in which bundle geometry surfaces decisive diagnostics unavailable to naive consensus.

1 Introduction

Multi-agent AI is moving rapidly from research curiosity to engineering infrastructure. Contemporary systems assign different subtasks to different language models, permit specialized tools or retrieval pathways for each agent, and then attempt to aggregate the resulting outputs into a single decision or artifact. The pattern appears across coding assistants, research agents, security analyzers, customer-support orchestrators, and autonomous browsing systems. Yet despite this practical importance, the core mathematical question is unsettled: *what does it mean for a team of AI agents to be globally consistent?* Put differently, under what conditions do local outputs produced under different evidence conditions, prompt contexts, trust ceilings, and tool permissions combine into a single reliable global section of work?

The dominant answers in the literature are mostly procedural. Majority vote is the most common baseline; more elaborate variants use self-consistency, chain-of-thought comparison, debate, or confidence-weighted ranking. These methods improve robustness in some regimes, but they remain essentially 0-dimensional. They compare endpoints without modeling the geometry of trust between them. If Agent A trusts Agent B because B used tools, and Agent B trusts Agent C because C sounds coherent, then A may end up indirectly granting C far more authority than C deserves. This is not merely an error in scoring; it is a failure of *transport*. The meaning of trust changes along the path by which a judgment moves through the system.

Judgment Geometry suggests a different answer. In the broader JuGeo program, judgments do not live in a flat list but on semantic sites equipped with covers, restriction maps, and descent conditions. Obstructions to gluing are detected cohomologically. That perspective already improves on naive voting because it asks whether local sections agree on overlaps. But even that is not enough for multi-agent systems. Sheaf cohomology detects whether claims glue; it does not, by itself, measure whether the *trust field* used to compare those claims is internally coherent. We therefore introduce a differential-geometric layer on top of the sheaf layer.

Trust is not a label. In most AI systems, trust is modeled as a scalar or categorical label attached to an output. We argue that this is the wrong primitive. Trust is better modeled as a *connection*: it specifies how a judgment’s trust level changes when that judgment is moved from one agent-task context to another. Once trust is modeled as a connection, familiar geometric notions become available. Curvature measures path dependence. Holonomy measures the total trust defect around a loop. Characteristic classes summarize global structure. Echo chambers, phantom consensus, cascades, and monopolies become geometric phenomena rather than heuristics.

The advantage is practical as well as conceptual. Bundle diagnostics tell us not only whether a team disagrees, but *what kind of team it is*. A team with $c_1 > 0$ exhibits trust inflation: mutually reinforcing weak evidence creates unjustified certainty. A team with $c_1 < 0$ exhibits trust deflation: strong evidence is repeatedly attenuated across boundaries, producing unnecessary conservatism or hidden single-agent dominance. A team with trivial pairwise disagreements but non-trivial holonomy is structurally unsound even if all local tests appear fine.

This paper makes five contributions.

1. We define the *judgment fiber bundle*, whose fibers consist of judgments (σ, e, τ, χ) above an agent-task site (section 3).
2. We define trust transport as a connection and prove discrete analogues of gauge invariance and the Ambrose–Singer relation between holonomy and curvature (sections 3 and 4).

3. We show that JuGeo’s existing Čech cohomological machinery embeds into the bundle framework, with H^1 contradictions corresponding to local curvature and H^2 cascades corresponding to holonomy (section 5).
4. We describe a concrete implementation in `jugeo-agents`, including adapters for `CLAUDE CODE`, `COPILLOT CLI`, and `CODEX`, together with web-stack trust bundles and cross-language bundle diagnostics (section 6).
5. We provide worked examples showing how bundle geometry rapidly exposes useful structure in coding and web verification tasks (sections 7 and 7.3).

The paper is self-contained, but it should be read as part of the JuGeo series: semantic sites provide the base category, trust algebra provides the ordered epistemic lattice, and descent provides the sheaf-theoretic gluing criterion. The judgment bundle adds a new statement: even when gluing data are available, there may still be a geometric obstruction in the trust field itself.

2 Preliminaries

2.1 Fiber bundles and connections

A fiber bundle consists of a total space $\mathcal{E}$, a base space $\mathcal{B}$, a projection map $\pi : \mathcal{E} \rightarrow \mathcal{B}$, and fibers $\mathcal{F}_p = \pi^{-1}(p)$ that are locally but not necessarily globally trivial. In our setting, the base space is discrete and combinatorial rather than smooth, but the bundle intuition remains useful: each point in the base determines a context, and the fiber above that point contains the judgments available in that context.

A connection ∇ specifies how to compare neighboring fibers. In the smooth setting it can be viewed as a horizontal sub-bundle or a covariant derivative; in the discrete setting used here it is more naturally a transport rule on edges of the base nerve. Given neighboring points $p, q \in \mathcal{B}$, the connection provides a parallel transport map $\nabla_{p \rightarrow q} : \mathcal{F}_p \rightarrow \mathcal{F}_q$ that adjusts trust according to the evidential relation between contexts. Curvature measures the failure of transport to be path-independent. Holonomy measures the net effect of transport around a closed loop.

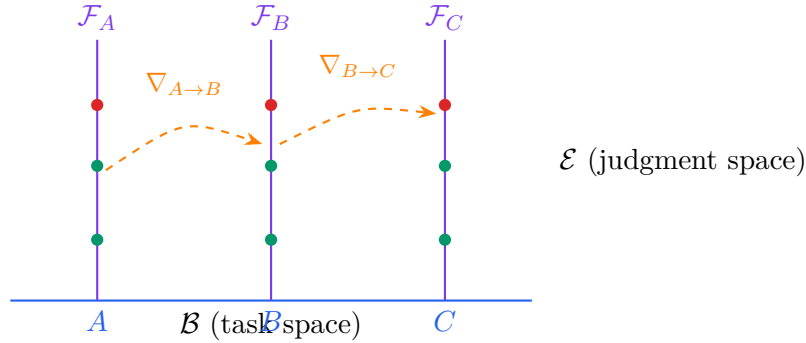


Figure 1: A discrete fiber bundle view of a multi-agent task: points in the base are agent-task contexts; fibers contain judgments in those contexts; dashed arrows represent trust transport.

The first Chern class is, in the smooth complex-line-bundle setting, the de Rham cohomology class represented by curvature. In our discrete setting we use an average-curvature surrogate that plays the same practical role: it summarizes whether a globally flat trust assignment exists. This

is not merely notation borrowing. The essential phenomenon is the same: local transport data can carry a global obstruction.

2.2 Čech cohomology and descent

Let $\mathcal{U} = \{U_i\}$ be a cover of a semantic site and let $\mathcal{S}$ be a sheaf of judgments or proposals on that site. A 0-cochain is a family of local sections $\{s_i \in \mathcal{S}(U_i)\}$. Its coboundary measures pairwise mismatch on overlaps $U_i \cap U_j$. A vanishing coboundary means the local sections satisfy the cocycle condition. When those compatible local sections glue to a global section, descent succeeds. When they do not, the obstruction appears in $\check{H}^1$ and its cohomology class records the failure.

JuGeo uses this structure operationally. Agent outputs are turned into local sections; overlaps are computed from task decompositions, shared claims, or cross-language contracts; contradiction detection yields H^1 -style obstructions; cascades and phantom consensus behave like higher obstructions. The present paper does not replace that sheaf machinery. It asks a second question: how coherent is the trust structure used to compare those local sections?

2.3 Trust algebra in Judgment Geometry

JuGeo does not treat trust as a scalar confidence score. It is an ordered algebra with explicit ceilings, promotion rules, demotion rules, and channel-sensitive constraints. In `jugeo-agents`, the canonical lattice is ordered as follows:

SELF_CONTRADICTED < UNGROUNDED_CLAIM < WEAK_MODEL_GENERATED < STRONG_MODEL_GENERATED

Three axioms matter for the present paper.

1. **No silent promotion.** Trust may increase only via explicit, named evidence routes.
2. **Conservative join.** Aggregating evidence cannot silently exceed the strongest admissible supporting channel.
3. **Evidence-gated promotion.** Each channel has a ceiling: a pure model response cannot become tool-verified without tool evidence.

These axioms already make trust algebraic. The judgment bundle makes it geometric by studying how trust changes under transport.

3 The judgment fiber bundle

3.1 The base space: the agent-task site

Definition 3.1 (Judgment). A *judgment* is a tuple

$$J = (\sigma, e, \tau, \chi)$$

where σ is a proposition or claim, e is a finite evidence set, $\tau \in \mathfrak{T}$ is a trust level, and χ is the channel through which the evidence was obtained.

Definition 3.2 (Agent-task site). Let $\mathcal{A} = \{a_1, \dots, a_m\}$ be a set of agents and $\mathcal{Q} = \{q_1, \dots, q_n\}$ a task decomposition. The *agent-task site* $\mathcal{B}$ is the nerve of the covering induced by assigning subtasks to agents. Its points are pairs $p = (a_i, q_j)$, and its edges connect points that overlap on shared specifications, files, interfaces, or claims. Higher simplices encode multi-way overlap.

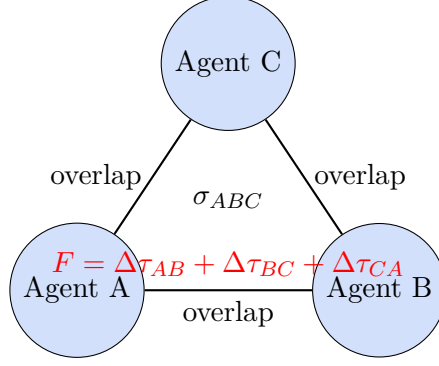


Figure 2: The nerve of an agent-task cover. Curvature is computed on the 2-simplex induced by three mutually overlapping agents.

3.2 Fibers and total space

Definition 3.3 (Judgment fiber). For each point $p \in \mathcal{B}$, the *judgment fiber* above p is

$$\mathcal{F}_p = \{(\sigma, e, \tau, \chi) : \text{the context } p \text{ produces a judgment about } \sigma\}.$$

Definition 3.4 (Judgment bundle). The *judgment bundle* is the projection

$$\pi : \mathcal{E} \rightarrow \mathcal{B},$$

where the total space is the disjoint union

$$\mathcal{E} = \coprod_{p \in \mathcal{B}} \mathcal{F}_p.$$

The structure group is the trust lattice acting by admissible transport on the trust coordinate of each judgment.

Intuitively, the bundle separates two kinds of structure that are often confounded in agent systems. The base remembers *where* a judgment came from: which agent, subtask, interface, or lifecycle stage. The fiber remembers *what* was judged and under what evidential conditions. This separation is what makes trust transport meaningful: trust changes not because the claim has changed, but because the context in which it is compared has changed.

$$\begin{array}{ccc} \mathcal{F}_A & \xrightarrow{\nabla_{A \rightarrow B}} & \mathcal{F}_B \\ \pi \downarrow & & \downarrow \pi \\ A & \longrightarrow & B \end{array} \tag{1}$$

3.3 Trust as a connection

Definition 3.5 (Trust connection). A *trust connection* on the judgment bundle assigns to each oriented edge $p \rightarrow q$ in the nerve of $\mathcal{B}$ a transport map

$$\nabla_{p \rightarrow q} : \mathcal{F}_p \rightarrow \mathcal{F}_q$$

that acts by transporting judgments with the same or overlapping semantic content while adjusting trust according to the evidential relationship between p and q .

Operationally, **jugeo-agents** constructs this connection from observed trust pairs. If Agent A makes a tool-verified claim and Agent B makes an otherwise matching citation-backed claim, then transport from A to B carries a negative trust delta. If a lower-trust agent's claim becomes stronger only after a tool check at the target, then transport may carry a positive delta. When no relevant comparison is available, the transport is taken to be flat.

Definition 3.6 (Curvature). For a 2-simplex (A, B, C) in the base nerve, the curvature is

$$\Omega(A, B, C) = \Delta_\tau(A \rightarrow B) + \Delta_\tau(B \rightarrow C) + \Delta_\tau(C \rightarrow A).$$

Non-zero curvature means trust transport around the triangle is path-dependent.

Theorem 3.7 (Gauge invariance). *Let all trust levels in a connected region be shifted by a constant c in the strength index representation. Then the curvature of every 2-simplex is unchanged.*

Proof. If the transported trust differential on an edge is $\Delta_\tau(A \rightarrow B) = \tau_B - \tau_A$, then after the gauge shift we have

$$\Delta'_\tau(A \rightarrow B) = (\tau_B + c) - (\tau_A + c) = \tau_B - \tau_A.$$

Therefore each edge differential is unchanged, and hence their sum around any triangle is unchanged as well. $\square$

Theorem 3.8 (Discrete Ambrose–Singer). *Let γ be a closed loop in the 1-skeleton of the base nerve, and let $\mathcal{T}(\gamma)$ be any triangulation of the region it bounds. Then*

$$\text{Hol}(\gamma) = \sum_{\sigma \in \mathcal{T}(\gamma)} \Omega(\sigma).$$

Proof. Write each face curvature as the oriented sum of its three edge transport values. Summing over all faces in the triangulation causes every interior edge to appear exactly twice with opposite orientation, so all interior contributions cancel. The only surviving terms are the boundary edges, which are precisely the transport values along the loop γ . Their sum is the holonomy. $\square$

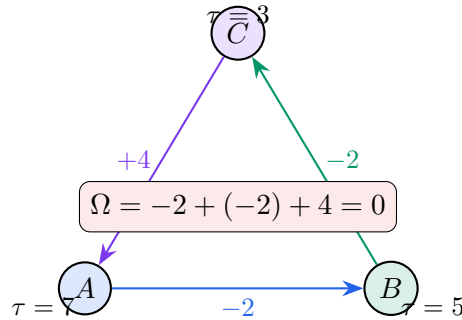


Figure 3: Holonomy around a three-agent loop. Even in a small team, trust transport may or may not close consistently.

4 Characteristic classes and structural pathologies

Definition 4.1 (First Chern class). Let $\mathcal{F}_2$ denote the set of 2-faces in the base nerve. The discrete first Chern class of the judgment bundle is the average curvature

$$c_1 = \frac{1}{|\mathcal{F}_2|} \sum_{\sigma \in \mathcal{F}_2} \Omega(\sigma).$$

Theorem 4.2 (Flatness criterion). *Suppose every connected component of the base nerve is triangulated and every edge transport is determined by admissible trust differentials. Then $c_1 = 0$ and trivial loop holonomy if and only if the judgment bundle admits a globally consistent flat trust assignment on that component.*

Proof. If the bundle admits a globally flat assignment, then every loop has trivial holonomy and every triangular face has zero curvature; therefore the average curvature is zero. Conversely, if the average curvature is zero and every loop holonomy is trivial, then edge transport is path-independent on the connected component. Fix a basepoint and assign its trust arbitrarily; path independence extends this assignment uniquely to every other point. The resulting global assignment realizes the observed transports without contradiction, so the connection is flat. $\square$

Corollary 4.3. *If $c_1 \neq 0$, then there exists no globally consistent trust assignment for the team. The unreliability is structural rather than local.*

The sign of c_1 carries actionable meaning.

Value of c_1	Interpretation	Team pathology
$c_1 = 0$	Flat bundle	Healthy team geometry
$c_1 > 0$	Trust inflation	Echo chamber or circular citation
$c_1 < 0$	Trust deflation	Over-attenuation or hidden monopoly
Large $ c_1 $	High curvature	Severe structural instability

These interpretations are visible in practice. Positive curvature indicates that trust increases as judgments circulate, usually because weak agents cite or echo each other until the team behaves as though independent confirmation has occurred. Negative curvature indicates repeated attenuation, often because one agent really has the only strong evidence while the others merely relay weaker versions of it. In both cases, the bundle gives a team-level diagnosis rather than merely ranking individuals.

5 Embedding Čech cohomology into bundle geometry

The bundle framework is designed to sit on top of JuGeo’s existing sheaf layer, not to replace it. The crucial relationship is that the failure of sections to glue in the sheaf appears as non-flat geometry in the bundle.

Theorem 5.1 (Embedding theorem). *There is a natural transformation from the obstruction calculus of the judgment sheaf to the curvature–holonomy calculus of the judgment bundle such that:*

1. H^0 global sections correspond to global flat sections of the bundle;
2. H^1 contradictions map to localized non-zero curvature on faces containing the contradictory overlap;
3. H^2 cascades map to non-trivial holonomy around loops induced by the cascade structure;
4. phantom consensus maps to flat local transport with evidentially empty or ungrounded fibers.

Proof. A compatible family of local sections defines both a sheaf-theoretic cocycle and a family of judgments in overlapping fibers. If the family glues globally, the transport values induced by matching judgments compose consistently, so the associated bundle section is flat. If a contradiction

appears on an overlap, then the induced transport around any face spanning that overlap cannot sum to zero, yielding curvature. Higher-order incompatibilities that evade pairwise checks require traversing loops in the overlap structure and therefore manifest as holonomy. Finally, phantom consensus consists of globally compatible but ungrounded claims; these induce flat transport because no explicit contradiction is present, yet the fibers remain epistemically empty. $\square$

$$\begin{array}{ccc}
\check{H}^1(\mathcal{U}, \mathcal{J}) & \xhookrightarrow{\iota} & \Omega^2(\mathcal{E}/\mathcal{B}) \\
\text{detects} \downarrow & & \downarrow \text{detects} \\
\text{contradictions} & \text{=====} & \text{non-zero curvature}
\end{array} \tag{2}$$

Proposition 5.2 (Strict generalization). *There exist structurally unhealthy multi-agent teams with trivial local contradiction sets but non-zero characteristic data. In particular, echo chambers and trust monopolies may be invisible to the raw Čech obstruction set while remaining visible through c_1 or non-trivial holonomy.*

Proof. Consider three agents that all repeat the same claim, so there is no pairwise contradiction and thus no H^1 witness. Suppose, however, that the claim is ungrounded and each agent’s trust is promoted only because the others repeat it. Then transport around the triangle inflates trust, producing positive curvature without an explicit contradiction. Likewise, if one agent has all the strong evidence and the others merely echo or attenuate it, pairwise consistency can still hold while loop transport records a non-trivial monopoly pattern. $\square$

The practical dictionary is therefore:

$$\begin{aligned}
H^0 &\leftrightarrow \text{verified global sections,} \\
H^1 &\leftrightarrow \text{curvature hotspots,} \\
H^2 &\leftrightarrow \text{holonomy defects,} \\
\text{phantom classes} &\leftrightarrow \text{flat but ungrounded fibers.}
\end{aligned}$$

6 Implementation in jugeo-agents

6.1 Architecture

The implementation is distributed across three layers. The shared schema layer contains trust levels, factual claims, contradictions, provenance structures, and agent-output dataclasses. The sheaf/cohomology layer contains claim extraction, contradiction detection, descent checking, and the global-section assembler. The bundle layer adds trust transport, curvature, holonomy, stratification, and characteristic classes. Adapters lift external systems into that common runtime.

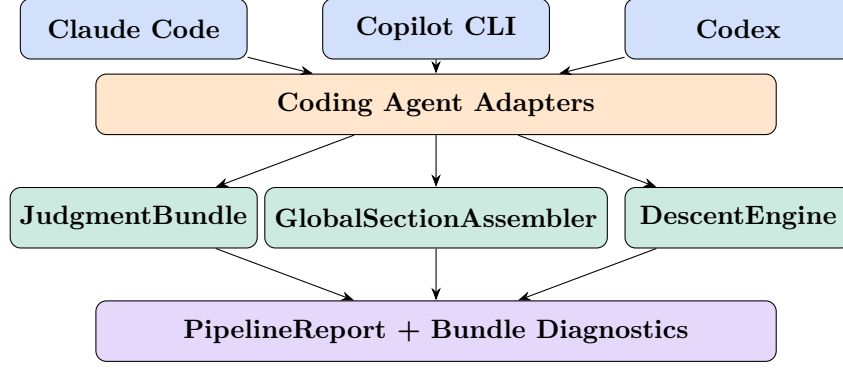


Figure 4: Implementation architecture in `jugeo-agents`. External coding agents are normalized into a common runtime, then analyzed simultaneously by descent, cohomology, and bundle geometry.

The recent retrofit performed across the JuGeo ecosystem realizes this design in three places.

1. In `jugeo-agents`, the wrapper and fusion pipelines now ingest every `AgentOutput` into `JUDGMENTBUNDLE`, and bundle diagnostics are attached to pipeline reports.
2. In the JuGeo core library, `jugeo.geometry.bundle` introduces a site-native `VerificationBundle` so the same ideas apply to semantic coordinates, proofs, tests, and theorem obligations.
3. In the web stack, `jugeo.webapp.trust.bundle`, `FiberedBundleDiagnostics`, and `DeviceBundleDiagnostics` bring connection geometry to request lifecycles, language boundaries, and cross-device visual invariants.

6.2 A coding-agents example: Claude Code, Copilot CLI, Codex

The most intuitive use-case is competitive coding. Three agents receive the same specification: implement a thread-safe token-bucket rate limiter in Python. They produce code under different evidence conditions. `CLAUDE CODE` uses tools to run tests. `COPILOT CLI` cites a plausible reference implementation but does not execute anything. `CODEX` produces a reasonable decorator from pure generation alone.

Listing 1: Competitive coding with three agent CLIs.

```

1 from jugeo_agents import (
2     JudgmentBundle,
3     GlobalSectionAssembler,
4     AgentOutput,
5     CodingAgentOrchestrator,
6 )
7
8 orch = CodingAgentOrchestrator(
9     task="Implement a thread-safe token bucket rate limiter in Python"
10 )
11
12 claude_output = orch.submit_agent_output(AgentOutput(
13     agent_id="claude-code",
14     model="claude-sonnet-4",
15     output_text=""

```

```

16 import threading
17 import time
18
19 class TokenBucket:
20     def __init__(self, rate: float, capacity: int):
21         self.rate = rate
22         self.capacity = capacity
23         self._tokens = capacity
24         self._last_refill = time.monotonic()
25         self._lock = threading.Lock()
26
27     def _refill(self) -> None:
28         now = time.monotonic()
29         elapsed = now - self._last_refill
30         self._tokens = min(self.capacity,
31                             self._tokens + elapsed * self.rate)
32         self._last_refill = now
33
34     def consume(self, tokens: int = 1) -> bool:
35         with self._lock:
36             self._refill()
37             if self._tokens >= tokens:
38                 self._tokens -= tokens
39                 return True
40             return False
41
42     """
43     tools_used=["file_write", "bash_run", "test_execute"],
44     tool_results={"test_execute": "All 12 tests passed"},
45 ))
46
47 copilot_output = orch.submit_agent_output(AgentOutput(
48     agent_id="copilot-cli",
49     model="gpt-4.1",
50     output_text=""
51 ))
52
53 import time
54
55 class RateLimiter:
56     def __init__(self, max_requests, window_seconds):
57         self.max_requests = max_requests
58         self.window_seconds = window_seconds
59         self.requests = []
60
61     def allow(self):
62         now = time.time()
63         self.requests = [t for t in self.requests
64                           if now - t < self.window_seconds]
65         if len(self.requests) < self.max_requests:
66             self.requests.append(now)
67             return True
68         return False
69
70     """
71     citations=["https://example.com/token-bucket-reference"],
72 ))
73

```

```

70 codex_output = orch.submit_agent_output(AgentOutput(
71     agent_id="codex",
72     model="codex-mini",
73     output_text=""
74 import time
75
76 def rate_limit(func, max_calls=100, period=60):
77     calls = []
78     def wrapper(*args, **kwargs):
79         now = time.time()
80         calls[:] = [c for c in calls if now - c < period]
81         if len(calls) >= max_calls:
82             raise Exception("Rate limited")
83         calls.append(now)
84         return func(*args, **kwargs)
85     return wrapper
86     """
87 ))
88
89 result = orch.verify()
90 print(result.bundle_summary)

```

A representative bundle report is:

Listing 2: Representative bundle diagnostic from the coding-agents example.

```

1  === Judgment Fiber Bundle Diagnostic ===
2  Agents: claude-code, codex, copilot-cli
3  Total judgments: 23
4
5  Connection Geometry:
6  First Chern class c1 = -1.3333
7  Trust deflation: tool-verified evidence systematically dominates.
8  Curved faces: 1 / 1
9  Bundle is flat: No
10
11 Team Holonomy:
12 Loop: claude-code -> codex -> copilot-cli
13 Total shift: -8.00
14 Non-trivial holonomy: hidden trust monopoly by claude-code
15
16 Trust Stratification:
17 UNGROUNDED_CLAIM: 8 judgments
18 CITATION_BACKED: 7 judgments
19 TOOL_VERIFIED: 8 judgments
20
21 Recommendation: trust claude-code's implementation.

```

Two things are important here. First, the decision is not based on a vague sense that one model is “better.” It is based on evidence channels and the resulting transport geometry. Second, the diagnosis is more expressive than a winner-takes-all leaderboard. The bundle tells us that the team is not a flat consensus but a negatively curved configuration dominated by one strong source. That makes the right operational response obvious: trust the winning code, quarantine the weak variants, and avoid pretending that a broad consensus has been established.

6.3 Knowledge fusion example

Bundle geometry also works for research-style teams that produce natural-language claims rather than code. Here the bundle is paired with sheaf-based claim extraction and the global section assembler.

Listing 3: Knowledge fusion with cohomology plus bundle diagnostics.

```
1 from jugeo_agents import GlobalSectionAssembler
2
3 assembler = GlobalSectionAssembler()
4 for output in research_team_outputs:
5     assembler.ingest(output)
6
7 result = assembler.assemble()
8 print("verified:", len(result.verified_claims))
9 print("quarantined:", result.quarantined_count)
10 print("metadata keys:", sorted(result.metadata.keys()))
11 print("bundle flat:",
        result.metadata["bundle_diagnostics"]["bundle_is_flat"])
```

The resulting object carries both sheaf data and geometric data. This matters because some failure modes are primarily logical while others are primarily geometric. A direct contradiction about the number of qubits in a hardware announcement is a sheaf-level issue first. A collection of ungrounded agents that all repeat the same market-size claim is geometric first: it forms a trust shape before it forms a contradiction.

6.4 A web-stack example

The web-specific bundles introduced in the retrofit make the same ideas useful far beyond agent-to-agent reasoning. In a full-stack Flask-style system, the request lifecycle

browser → route → handler → database → response → render

acts as a base path, while judgments about validation, sanitization, typing, authentication, and rendering populate the fibers above each stage.

Listing 4: Web trust bundle example.

```
1 from jugeo.webapp.trust.bundle import WebTrustBundle, WebJudgment
2
3 bundle = WebTrustBundle()
4 bundle.add_judgment(WebJudgment(
5     stage="browser",
6     language="javascript",
7     claim="email field validates format",
8     trust="CLIENT_VALIDATED",
9 ))
10 bundle.add_judgment(WebJudgment(
11     stage="handler",
12     language="python",
13     claim="email field validates format",
14     trust="SERVER_VALIDATED",
15 ))
16 bundle.add_judgment(WebJudgment(
```

```

17     stage="database",
18     language="sql",
19     claim="email column has NOT NULL constraint",
20     trust="DB_CONSTRAINT_ENFORCED",
21 ))
22 print(bundle.summary_text())

```

This generalization is not cosmetic. The same geometry that diagnoses a team of coding agents also diagnoses a cross-language web stack. That is exactly the kind of JuGeo unification we want: one theory of bundle transport that applies from semantic proofs to Flask handlers to CLI coding agents.

7 Worked examples

7.1 Echo chamber detection

Consider three agents A, B, and C. None has tool access. Each cites the other two as supporting evidence. The textual claims match, so naive majority vote accepts the consensus. Pairwise contradiction detection also remains silent. Yet the consensus is structurally illegitimate because all evidence is circular. In the bundle this appears as positive curvature.

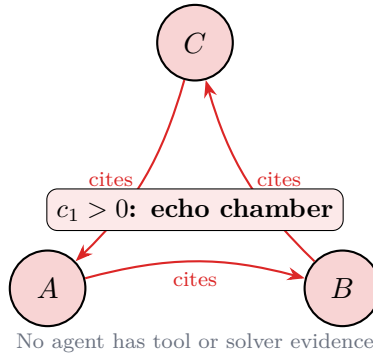


Figure 5: Positive curvature as echo chamber: apparent consensus without any independent grounding.

The appropriate intervention is not “pick the best wording” but “demand a new evidence channel.” Curvature tells the orchestrator to route the disputed claim to tools, retrieval, or formal proof rather than to more debate.

7.2 Trust monopoly detection

Now suppose one agent really does have the only strong evidence. The other two agree with it, but only after attenuation through weaker channels. Pairwise agreement looks reassuring. The bundle, however, records a negative holonomy: transport around the team loses trust in a way that reveals the team’s effective rank structure. This is not an echo chamber but a disguised single-expert configuration. Again, the bundle helps the orchestrator respond correctly: trust the strong agent, but do not misreport the result as broad consensus.

7.3 Cross-language web verification

The same geometry becomes particularly compelling in web applications because the most expensive bugs live in overlaps between layers rather than within any single language. Frontend code may validate a field that the backend fails to check; a Flask handler may assume non-null database values that the schema does not enforce; HTML may assume an authorization state that middleware never proves. Judgment Geometry treats these as descent failures. Bundle geometry refines the picture by measuring the trust transport across the browser–server–database loop.

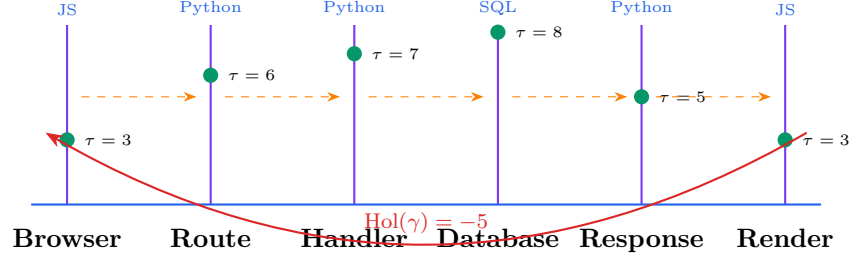


Figure 6: A web request lifecycle as a trust bundle. Cross-language transitions cause trust loss that accumulates around the lifecycle loop.

When the web bundle reports non-trivial holonomy, the message is immediate and useful: trust is leaking across boundaries. The system may be locally tested at each stage, yet still globally unreliable because the evidence does not transport cleanly through the full loop.

8 Trust stratification and the action principle

Bundle geometry naturally induces a stratification by trust level. Judgments at `TOOL_VERIFIED` form a different epistemic stratum from judgments at `CITATION_BACKED` or `UNGROUNDED_CLAIM`. This matters because contradictions within a stratum are more severe than contradictions across strata. If two tool-verified agents disagree, there is no easy trust-based tiebreaker. If a tool-verified agent disagrees with an ungrounded one, the geometry is already telling us which stratum should dominate.

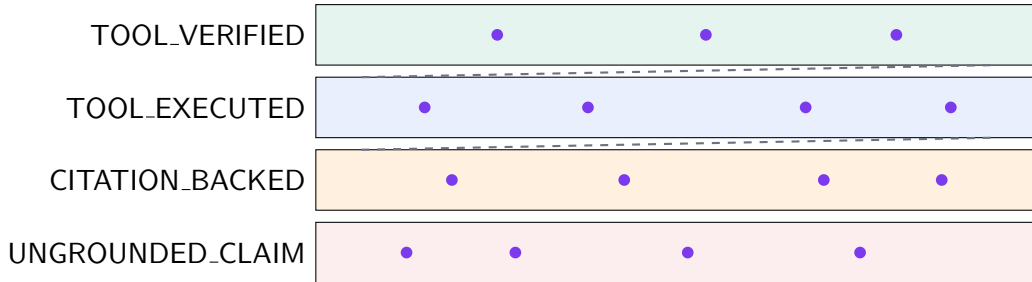


Figure 7: Trust stratification of the judgment bundle. Higher strata dominate lower ones in fusion, while intra-stratum contradictions are maximally serious.

This stratification leads naturally to an action principle for verification. A *semantic move* is any operation that changes the bundle: challenge an agent, run a tool, request a citation, launch a

proof search, or synthesize a treaty. Let the action of a move be

$$S(m) = \text{cost}(m) - \Delta\mathfrak{T}(m),$$

where $\Delta\mathfrak{T}(m)$ is the trust improvement produced by the move. Verification then becomes a geodesic problem: choose the path of moves that minimizes total action while reaching the desired trust stratum. This gives Judgment Geometry a precise answer to a practical orchestration question: not just *what is wrong*, but *what is the cheapest principled next step*?

9 Why this helps ideate better systems

One of the surprising outcomes of the bundle perspective is that it helps with *design*, not only verification. If a proposed application architecture induces pathological curvature before implementation, then the design is already suspicious. This observation is especially powerful in full-stack and multi-agent settings.

For coding-agent systems, bundle-aware ideation favors architectures in which strong channels are intentionally distributed rather than concentrated. A team of three nearly identical weak agents is geometrically boring and often useless. A team in which one agent owns all tool access and the others only summarize it may work, but it forms a monopoly geometry and therefore offers little redundancy. The most promising designs are those in which different agents possess different strong channels whose overlaps can actually corroborate one another.

For web applications, bundle-aware ideation suggests looking for app ideas whose value comes from maintaining invariants across boundaries. A mundane CRUD app has little geometric interest. By contrast, a Flask system that continuously verifies visually stable public dashboards across device classes, or that tracks policy conformance across browser, server, and database layers, has rich boundary structure and therefore benefits directly from JuGeo. This is one path toward novel useful Flask apps: target domains where correctness lives in the intersections and where the product itself can expose its own descent and bundle diagnostics to users.

10 Related work

The closest adjacent work comes from several directions, none of which provides a comparable geometric account. Debate-style AI safety protocols [6] use adversarial comparison to improve truthfulness, but do not model trust transport. Reflection and self-consistency methods improve local reasoning, but again remain pointwise. Multi-agent orchestration systems such as AutoGen [7], MetaGPT [8], and CrewAI [9] provide engineering frameworks for agent composition, but not a formal semantics of global consistency.

Closer in spirit are attribution and coalition analyses based on Shapley values or influence scores; these quantify contribution but not path-dependent trust. Formal methods for distributed systems, proof-carrying code, and type systems also contribute relevant ideas, but typically reason about program states rather than epistemic transport between evidence-bearing judgments. The contribution of Judgment Geometry is precisely to combine the sheaf-theoretic language of local and global compatibility with the differential-geometric language of connection, curvature, and holonomy.

11 Conclusion and future work

We have argued for a simple but powerful reframing: in multi-agent AI systems, trust is not a score but a connection. Once that move is made, a broad range of otherwise slippery phenomena become computable. Echo chambers become positive curvature. Hidden monopolies become holonomy. Flat-but-ungrounded consensus becomes a phantom region. Existing JuGeo cohomology becomes the logical shadow of a richer geometric structure.

The implementation work already shows that this perspective scales across the JuGeo ecosystem. The same ideas now appear in the core site verifier, the multi-agent orchestration runtime, and the web verification stack. In every case, bundle geometry improves the system because it identifies structure that local checks miss.

Several directions are immediately open. Higher Chern classes could capture more subtle topologies in large agent teams. Spectral sequences could model convergence over many rounds of interaction. A Yang–Mills-like optimization principle could be used to learn or synthesize optimal trust connections under cost constraints. And from an engineering perspective, bundle-aware routing and repair policies may become one of the most practical ways to make AI agents both more autonomous and more trustworthy.

A Implementation notes

The concrete implementation added during this work includes the following artifacts.

- `jugeo_agents.core.bundle.JudgmentBundle`: computes trust connections, curvature, holonomy, the first Chern class, and trust stratification over agent outputs.
- `jugeo_agents.core.fusion.GlobalSectionAssembler`: now carries bundle diagnostics alongside sheaf-theoretic fusion data.
- `jugeo_agents.wrapper.JuGeoAgentWrapper`: now accumulates bundle diagnostics across the full pipeline and exposes them in `PipelineReport`.
- `jugeo.geometry.bundle.VerificationBundle`: brings the same construction to semantic-site verification in the JuGeo core library.
- `jugeo.webapp.trust.bundle.WebTrustBundle` plus `FiberedBundleDiagnostics` and `DeviceBundleDiagnostics`: extends connection geometry to request lifecycles, language fibers, and visual invariants across devices.

These modules collectively make bundle geometry a first-class concept across the entire ecosystem rather than a standalone research note.

References

- [1] Dale Husemoller. *Fibre Bundles*. Springer, third edition, 1994.
- [2] John Milnor and James D. Stasheff. *Characteristic Classes*. Princeton University Press, 1974.
- [3] Raoul Bott and Loring W. Tu. *Differential Forms in Algebraic Topology*. Springer, 1982.
- [4] Mark Goresky and Robert MacPherson. *Stratified Morse Theory*. Springer, 1988.

- [5] Michael Artin, Alexandre Grothendieck, and Jean-Louis Verdier. *Théorie des topos et cohomologie étale des schémas (SGA 4)*. Lecture Notes in Mathematics, Springer, 1972–1973.
- [6] Geoffrey Irving, Paul Christiano, and Dario Amodei. AI safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.
- [7] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [8] Sirui Hong, Xinlin Zheng, Jonathan Chen, et al. MetaGPT: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- [9] CrewAI. CrewAI framework documentation. <https://github.com/crewAIInc/crewAI>, accessed 2026.
- [10] LangChain. LangGraph documentation. <https://github.com/langchain-ai/langgraph>, accessed 2026.
- [11] JuGeo Research Group. Judgment Geometry paper series. Internal series including semantic sites, trust algebra, descent obstructions, fleet competition, and agent verification, 2024–2026.
- [12] Saunders Mac Lane. *Categories for the Working Mathematician*. Springer, second edition, 1998.
- [13] David I. Spivak. *Category Theory for the Sciences*. MIT Press, 2014.